Inżynieria oprogramowania - system dostarczania paczek

Krzysztof Jura, Konrad Kmiecik, Mateusz Kruk
Informatyka i Systemy Inteligentne, 2 rok

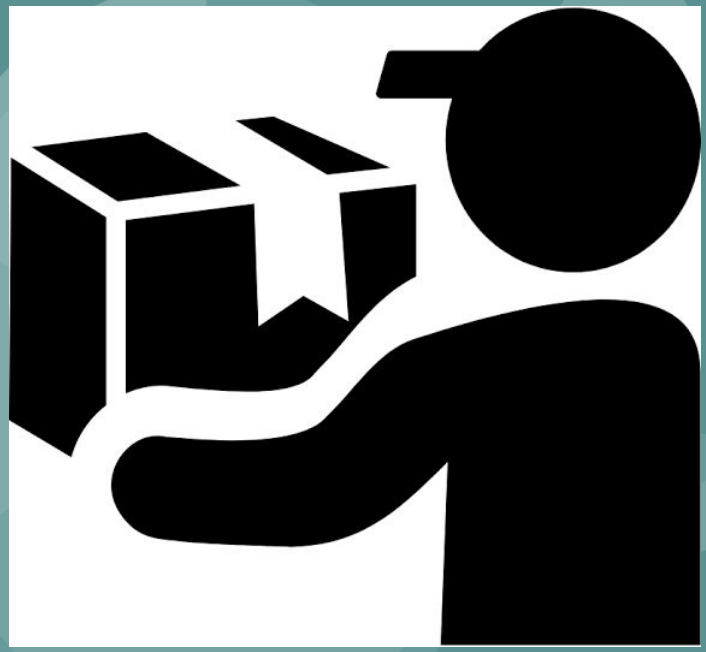
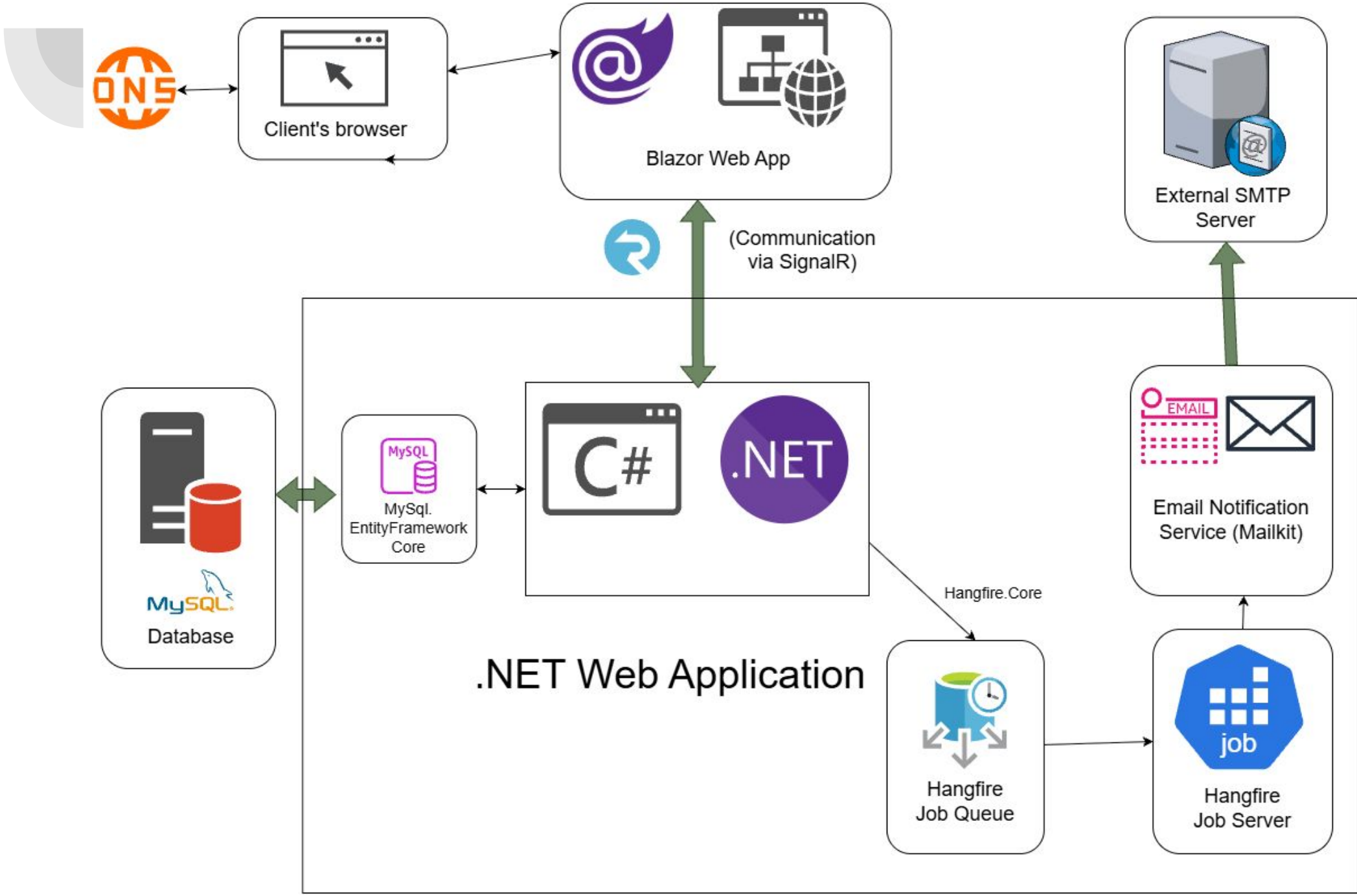




Diagram Architektury



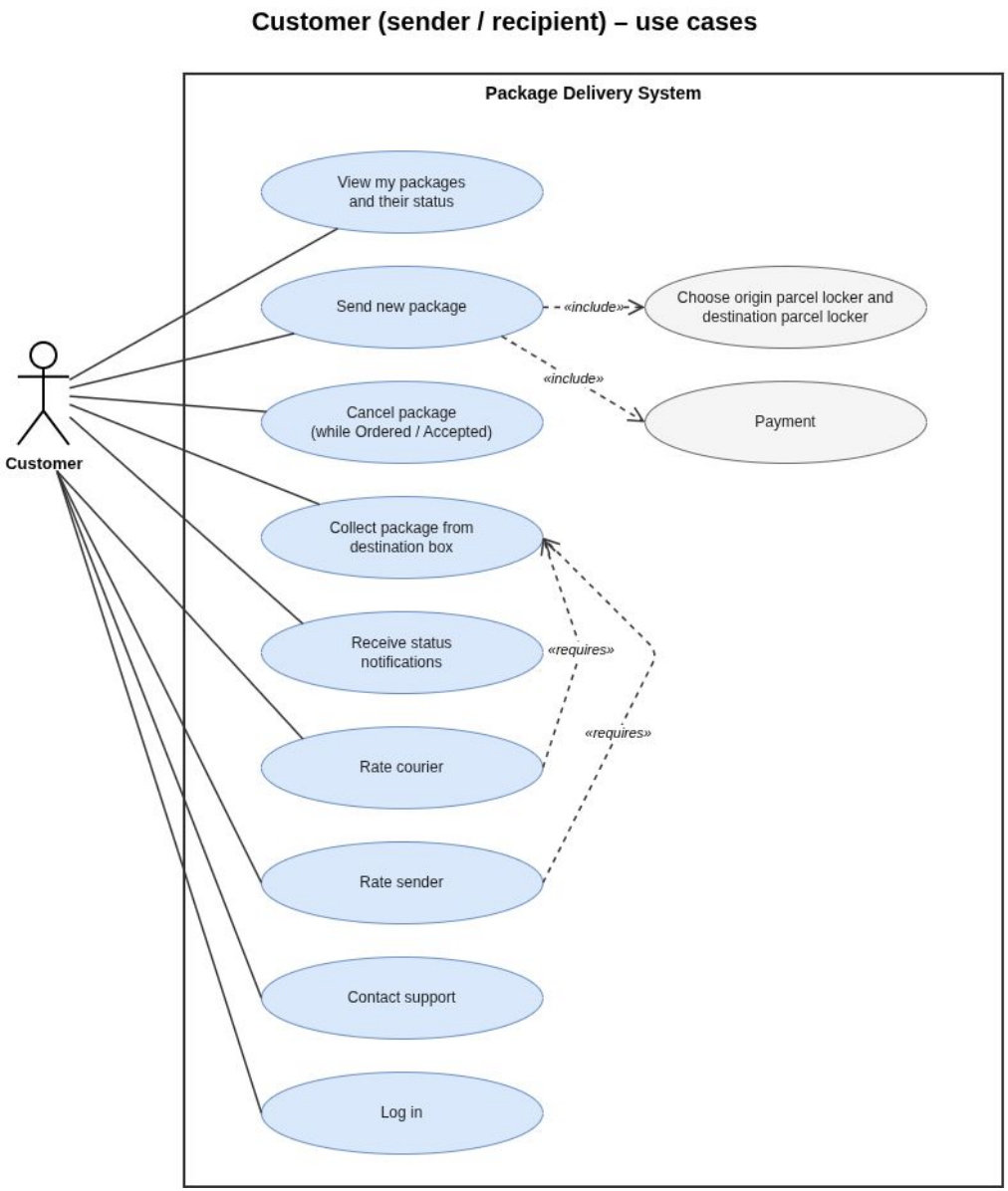
Stos technologiczny

Warstwa	Technologia	Wersja	Uzasadnienie/opis
Front-end	Blazor 	-	Zastosowanie frameworka Blazor pozwala na budowę UI aplikacji w języku C#, co unifikuje technologicznie frontend z backendem. Widoczna na diagramie dwukierunkowa komunikacja przez SignalR jest kluczowa dla systemu kurierskiego – umożliwia natychmiastowe przesyłanie współrzędnych GPS oraz aktualizowanie statusów paczek widocznych na stronie przez użytkownika.
Back-end	.NET 	10.0	Stanowi serce całego systemu widoczne w centralnej części schematu. Odpowiada za realizację skomplikowanej logiki biznesowej, w tym automatu stanów zamówień (od złożenia, przez przygotowanie, aż po dostawę) oraz autoryzację opartą na rolach. Modularna struktura .NET, wraz z obszerną dokumentacją, zapewnia stabilność, bezpieczeństwo oraz łatwość wstrzykiwania zależności (Dependency Injection).
Back-end	MySql.EntityFrameworkCore	10.0.9	Oficjalny dostawca ORM (Object-Relational Mapping) dla bazy MySQL w ekosystemie .NET. Umożliwia operowanie na danych w postaci obiektów C# zamiast czystego SQL, zapewnia wbudowane mechanizmy bezpieczeństwa (ochrona przed SQL Injection) oraz wygodne zarządzanie strukturą bazy poprzez system migracji.

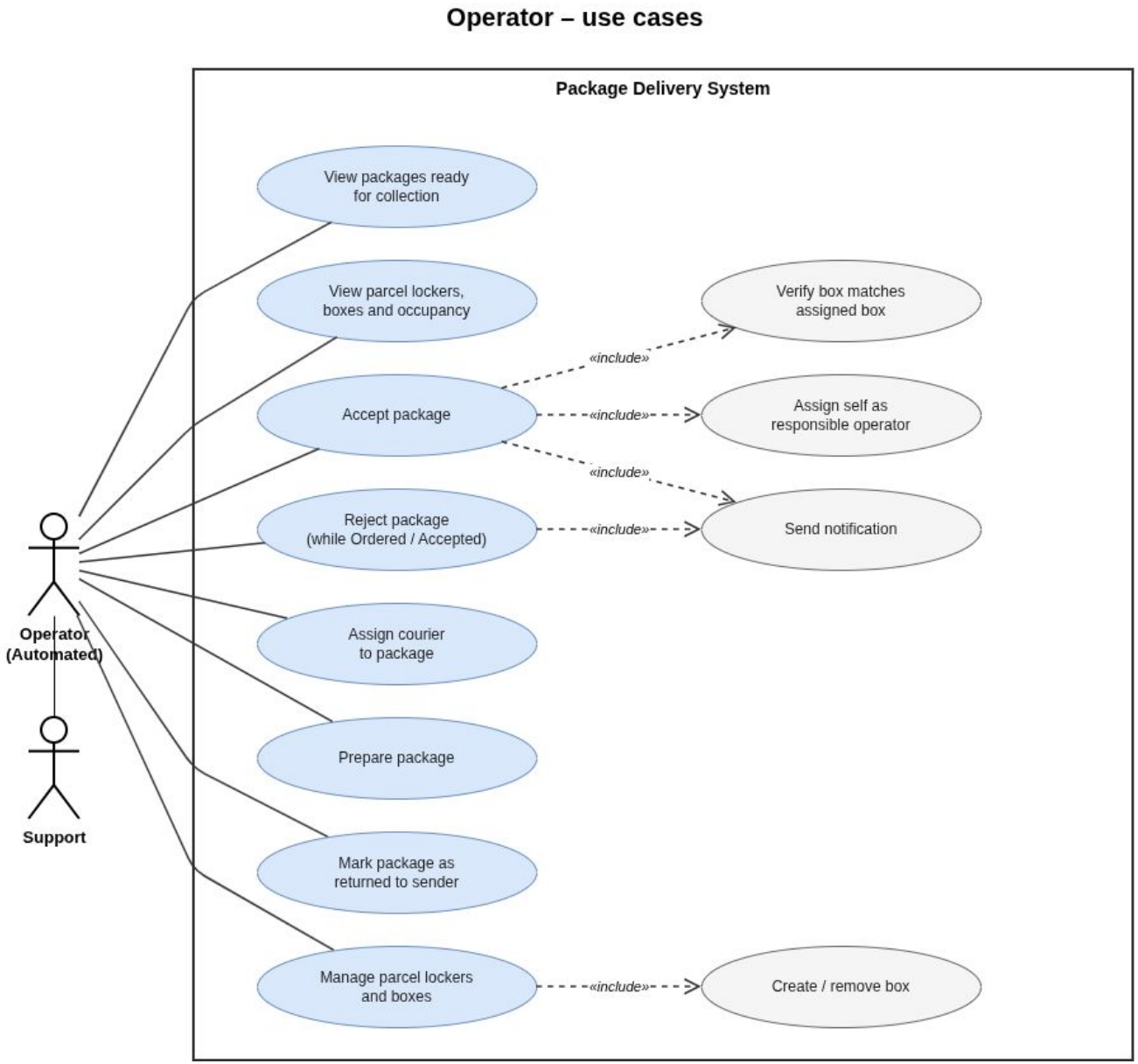
Stos technologiczny (c.d.)

Warstwa	Technologia	Wersja	Uzasadnienie/opis
Database	MySQL 	26.10.0	Wydajny, relacyjny serwer bazodanowy, doskonale radzi sobie z obsługą operacji transakcyjnych (OLTP), co jest kluczowe przy dynamicznych zmianach statusów zamówień oraz ciągłym napływie danych o lokalizacji.
Back-end	System.Security.Cryptography.Pkcs	10.0.13	Przestrzeń nazw umożliwiająca korzystanie z Public Key Cryptography Standards (PKCS). Wykorzystana zostanie do zabezpieczenia i podpisywania tokenów autoryzacyjnych operatorów, co zwiększa bezpieczeństwo systemu.
Back-end	Hangfire.Core 	1.8.25	Rozwiązuje problem wydajnościowy. Operacje takie jak masowa wysyłka powiadomień e-mail są kosztowne czasowo – dzięki Hangfire zadania te są bezpiecznie kolejkowane i przetwarzane asynchronicznie w tle. Główny wątek aplikacji (.NET) nie czeka na odpowiedź serwera pocztowego, dzięki czemu interfejs dla kuriera i klienta działa płynnie, a w przypadku awarii serwera zadania nie giną.
Back-end	MailKit	4.18.1	Połączenie stabilnej biblioteki Mailkit z zewnętrznym serwerem SMTP zapewnia niezawodną dostarczalność wiadomości e-mail do klientów. Realizuje to bezpośrednio wymóg biznesowy powiadamiania stron o każdej zmianie statusu przesyłki, izolując jednocześnie procesy wysyłki poczty od głównej logiki biznesowej aplikacji.

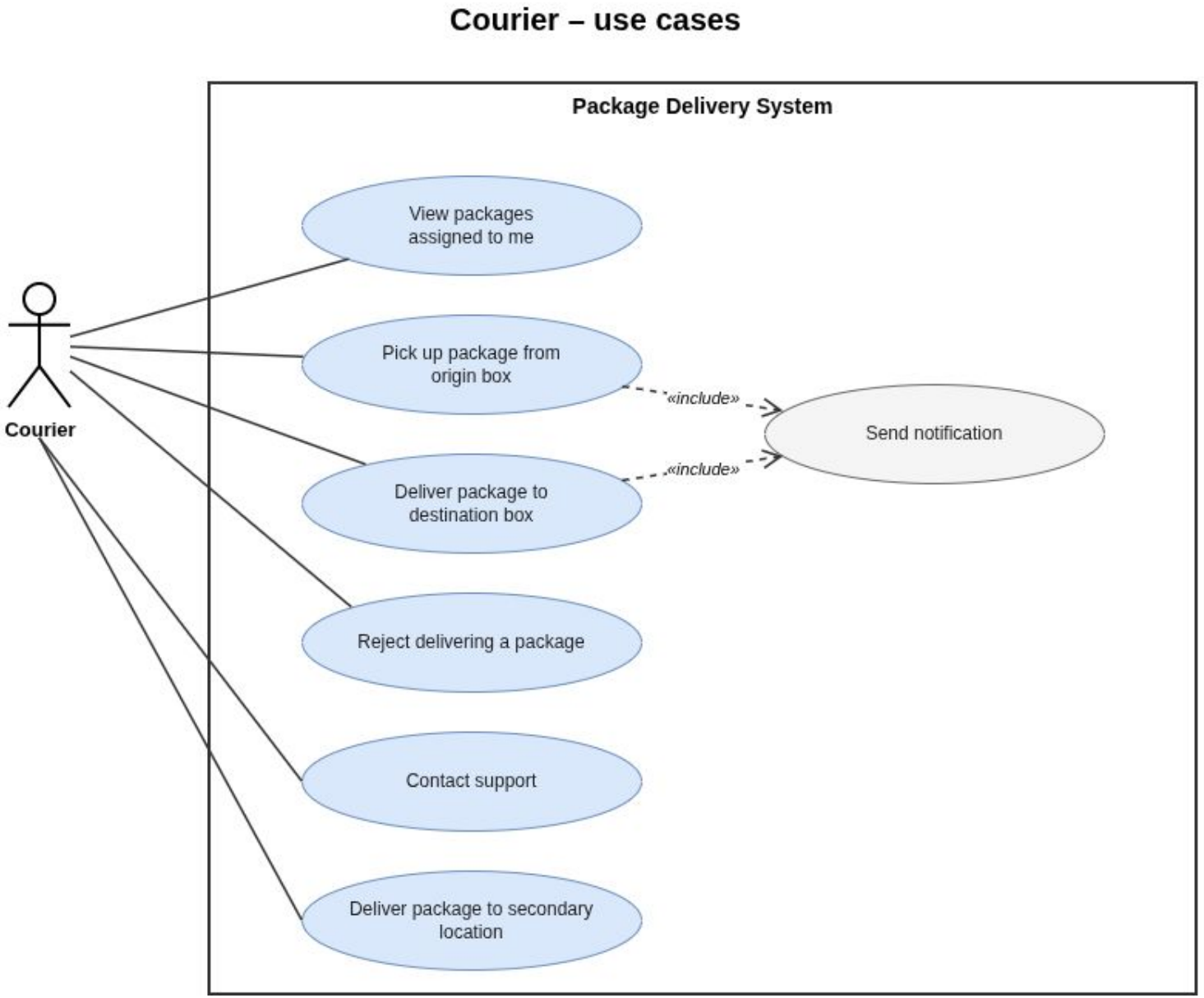
Use-cases


Customer (sender / recipient) – use cases
Package Delivery System
Customer
View my packages and their status
Send new package
«include»
Choose origin parcel locker and destination parcel locker
«include»
Payment
Cancel package (while Ordered / Accepted)
Collect package from destination box
Receive status notifications
«requires»
«requires»
Rate courier
Rate sender
Contact support
Log in

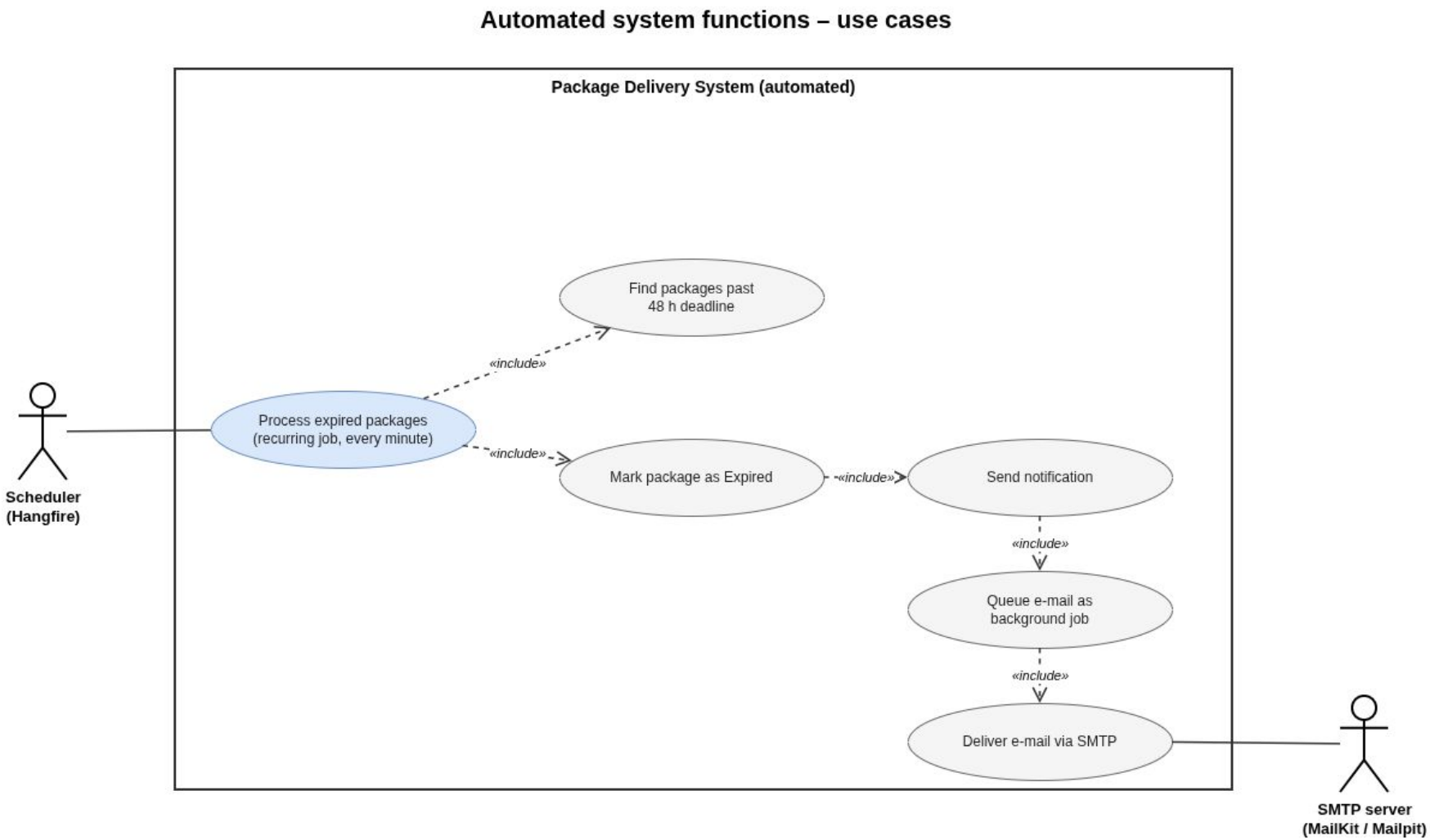
Use-cases


Operator – use cases
Package Delivery System
View packages ready for collection
View parcel lockers, boxes and occupancy
Accept package
Reject package (while Ordered / Accepted)
Assign courier to package
Prepare package
Mark package as returned to sender
Manage parcel lockers and boxes
Verify box matches assigned box
Assign self as responsible operator
Send notification
Create / remove box
«include»
«include»
«include»
«include»
«include»
Operator (Automated)
Support

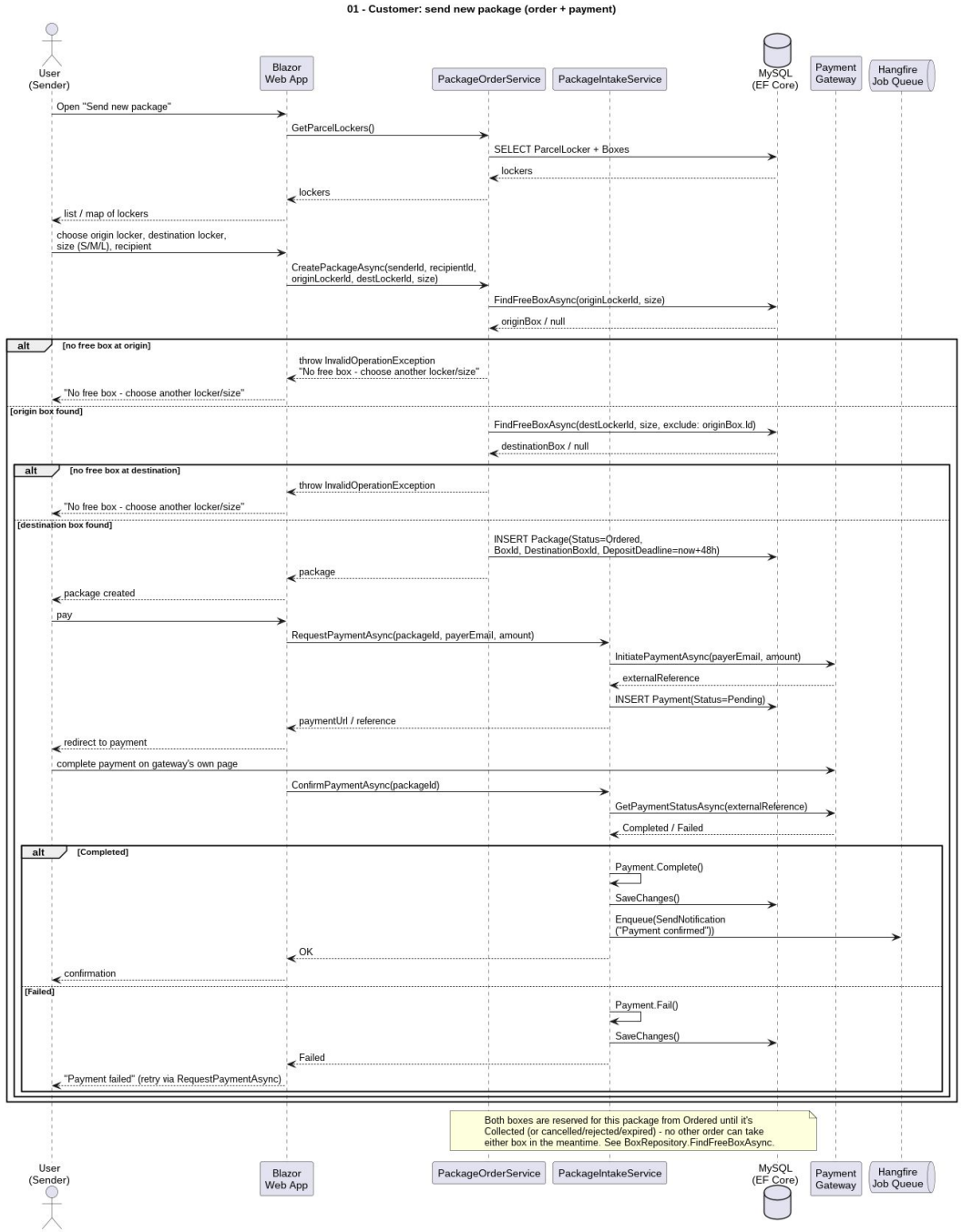
Use-cases


Courier – use cases
Package Delivery System
View packages assigned to me
Pick up package from origin box
«include»
Send notification
Deliver package to destination box
«include»
Reject delivering a package
Contact support
Deliver package to secondary location
Courier

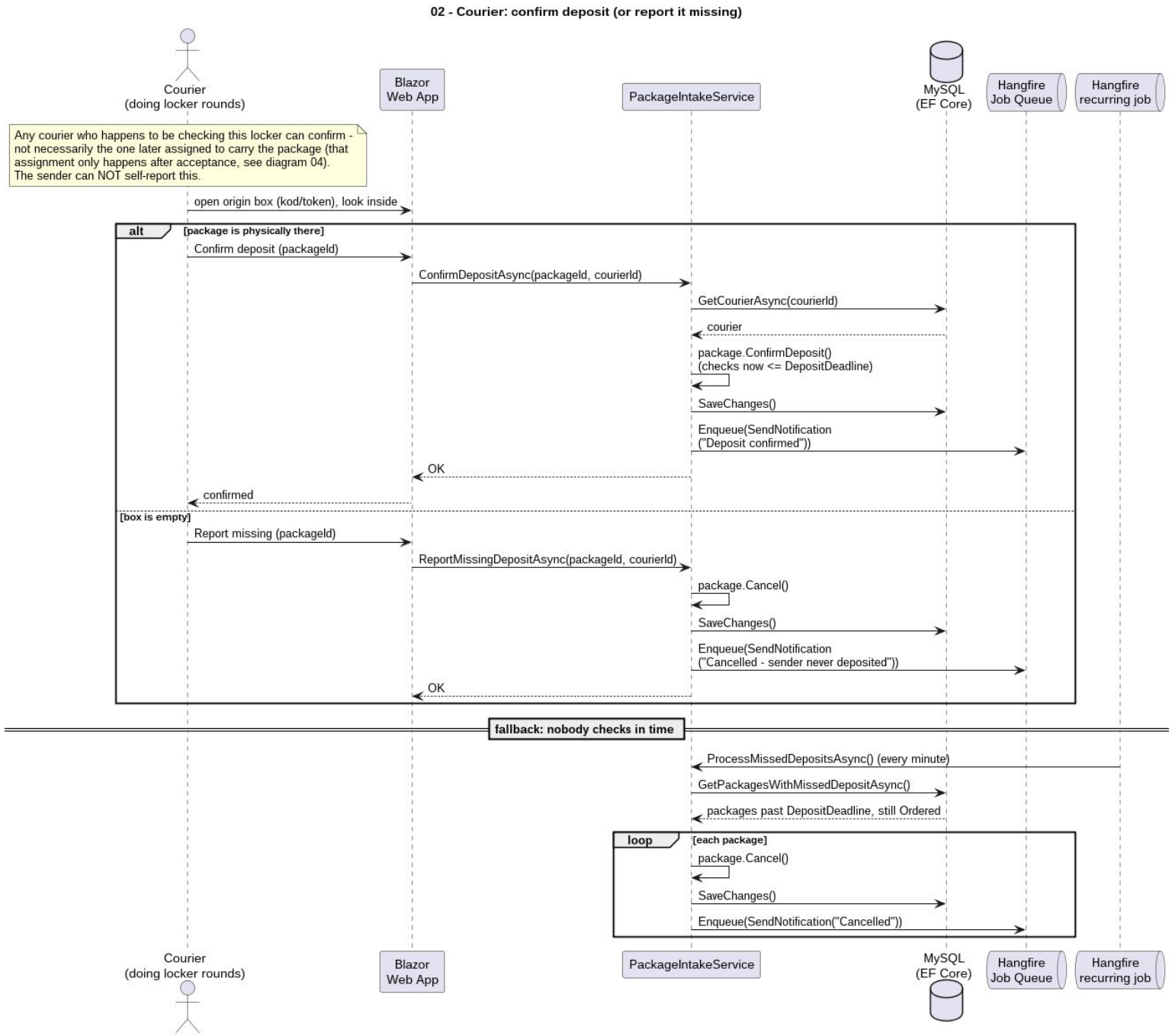
Use-cases


Automated system functions – use cases
Package Delivery System (automated)
Find packages past
48 h deadline
«include»
Process expired packages
(recurring job, every minute)
«include»
Mark package as Expired
«include»
Send notification
«include»
Queue e-mail as
background job
«include»
Deliver e-mail via SMTP
Scheduler
(Hangfire)
SMTP server
(MailKit / Mailpit)

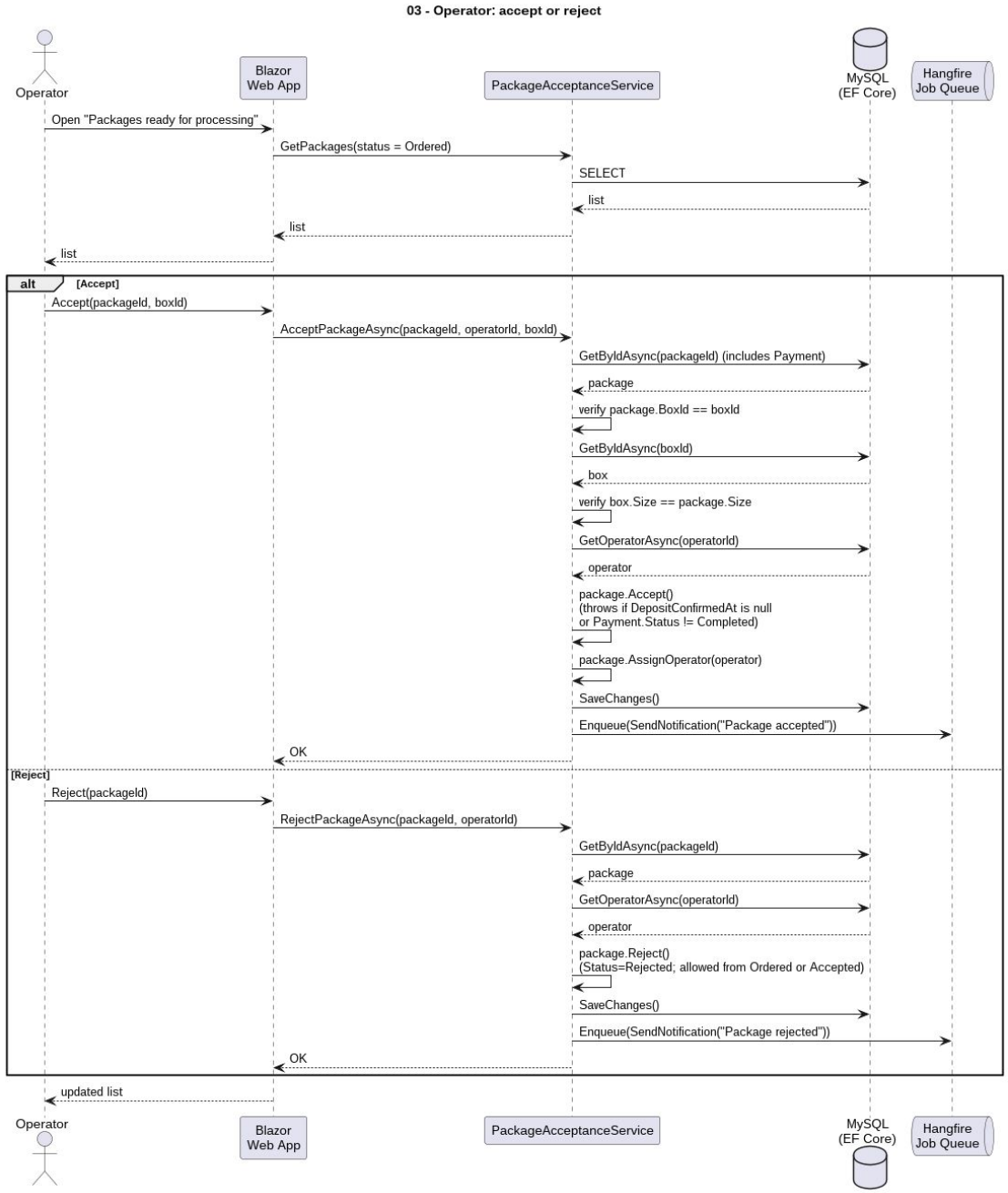
Diagramy sekwencji



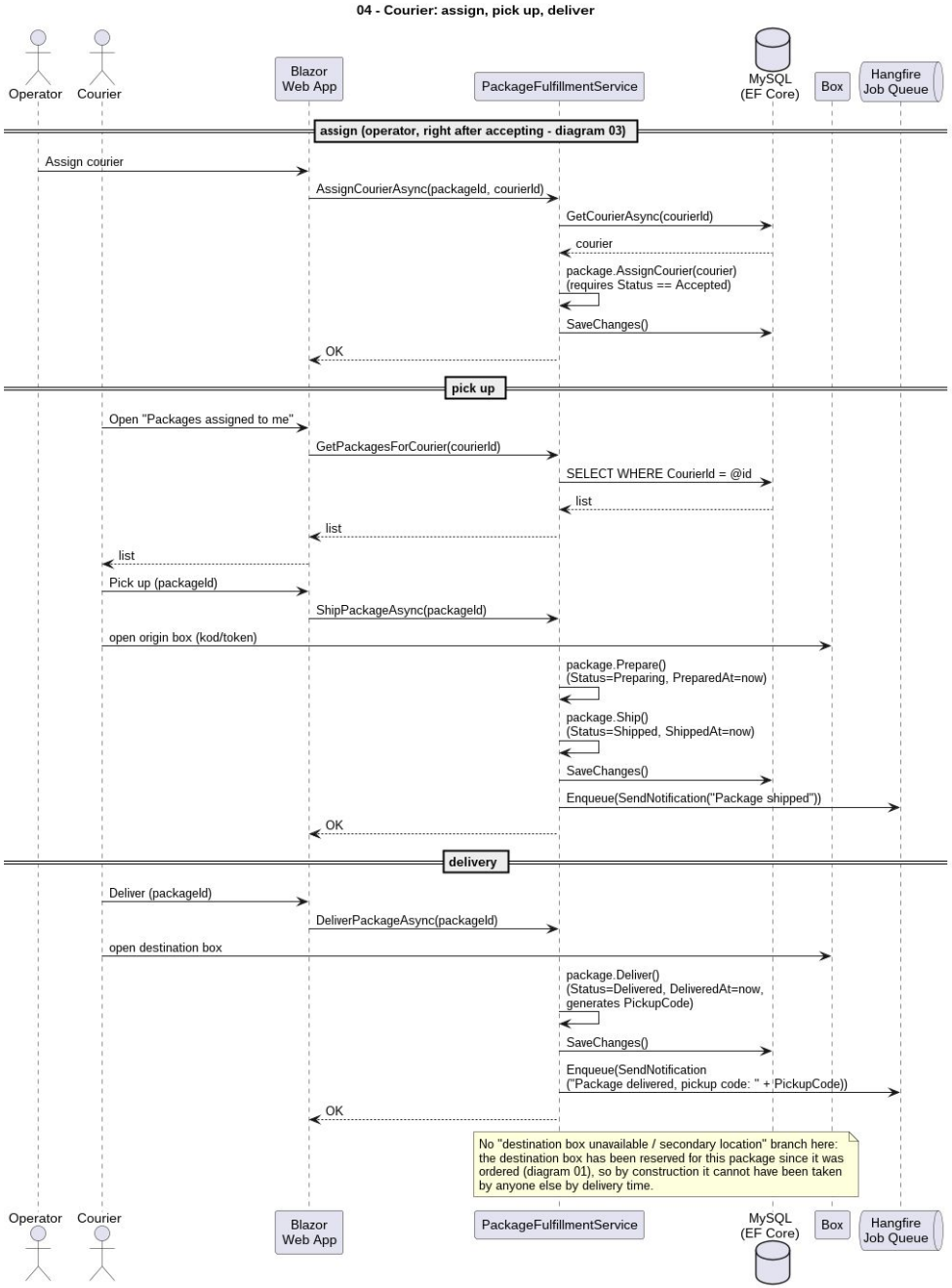
Diagramy sekwencji



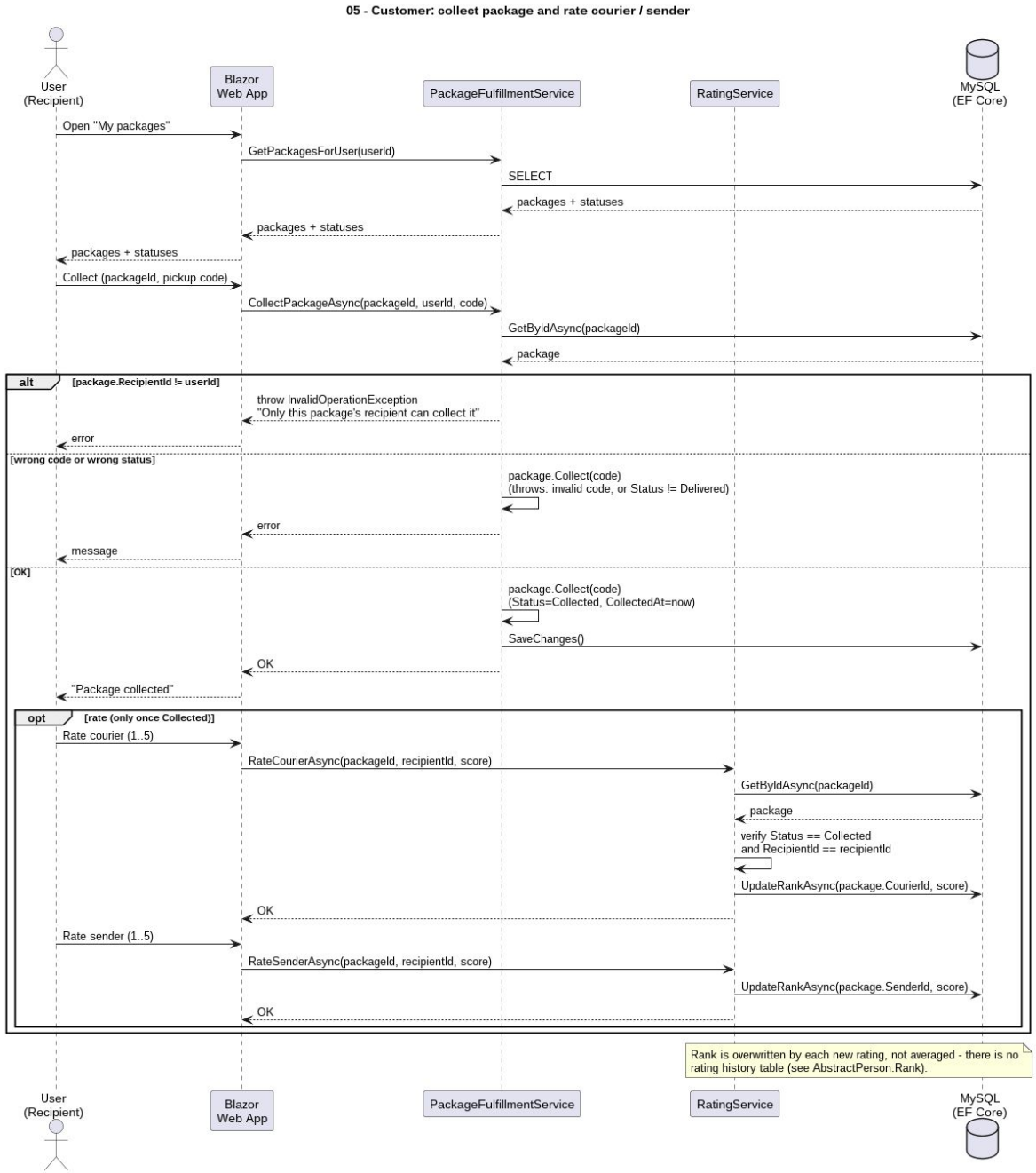
Diagramy sekwencji



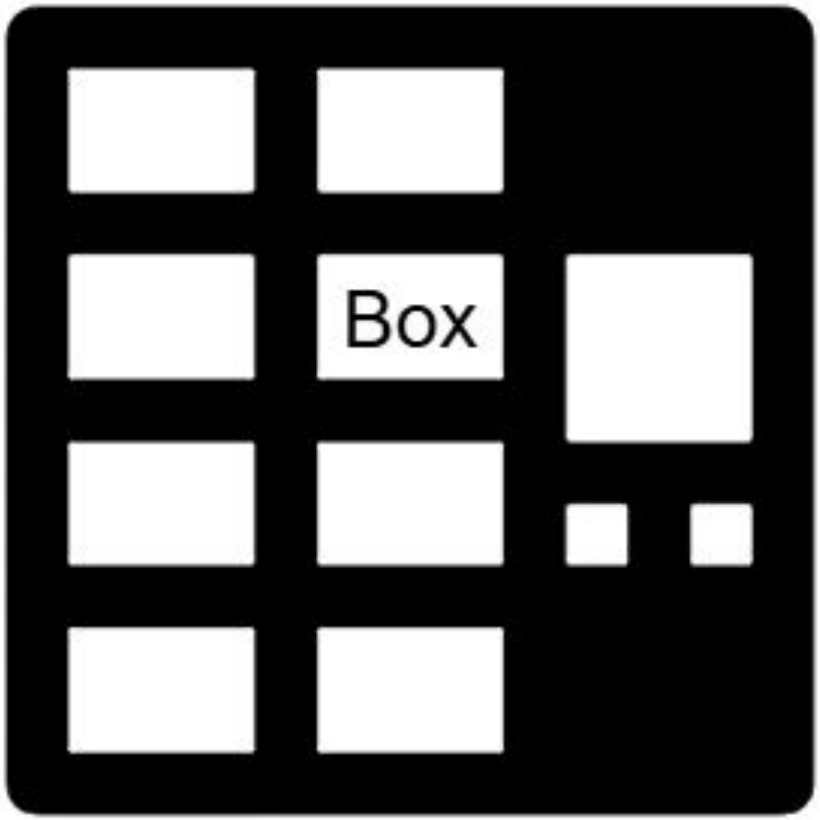
Diagramy sekwencji



Diagramy sekwencji



Schemat oznaczeń



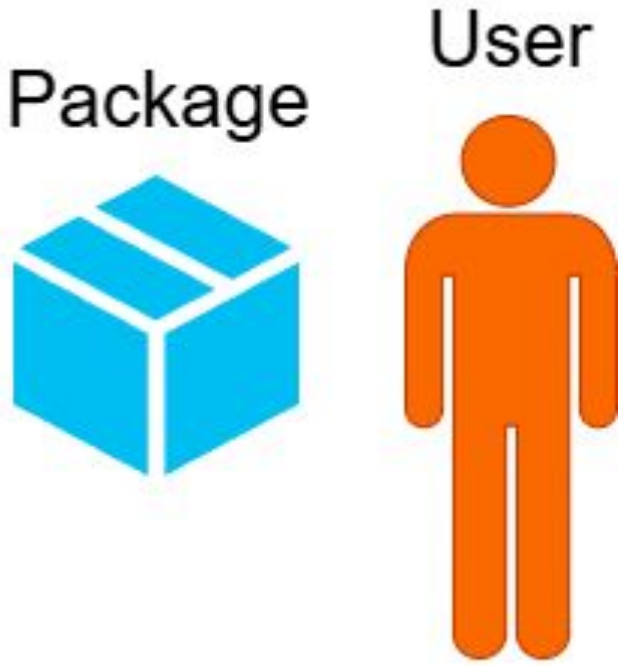


Diagram ER

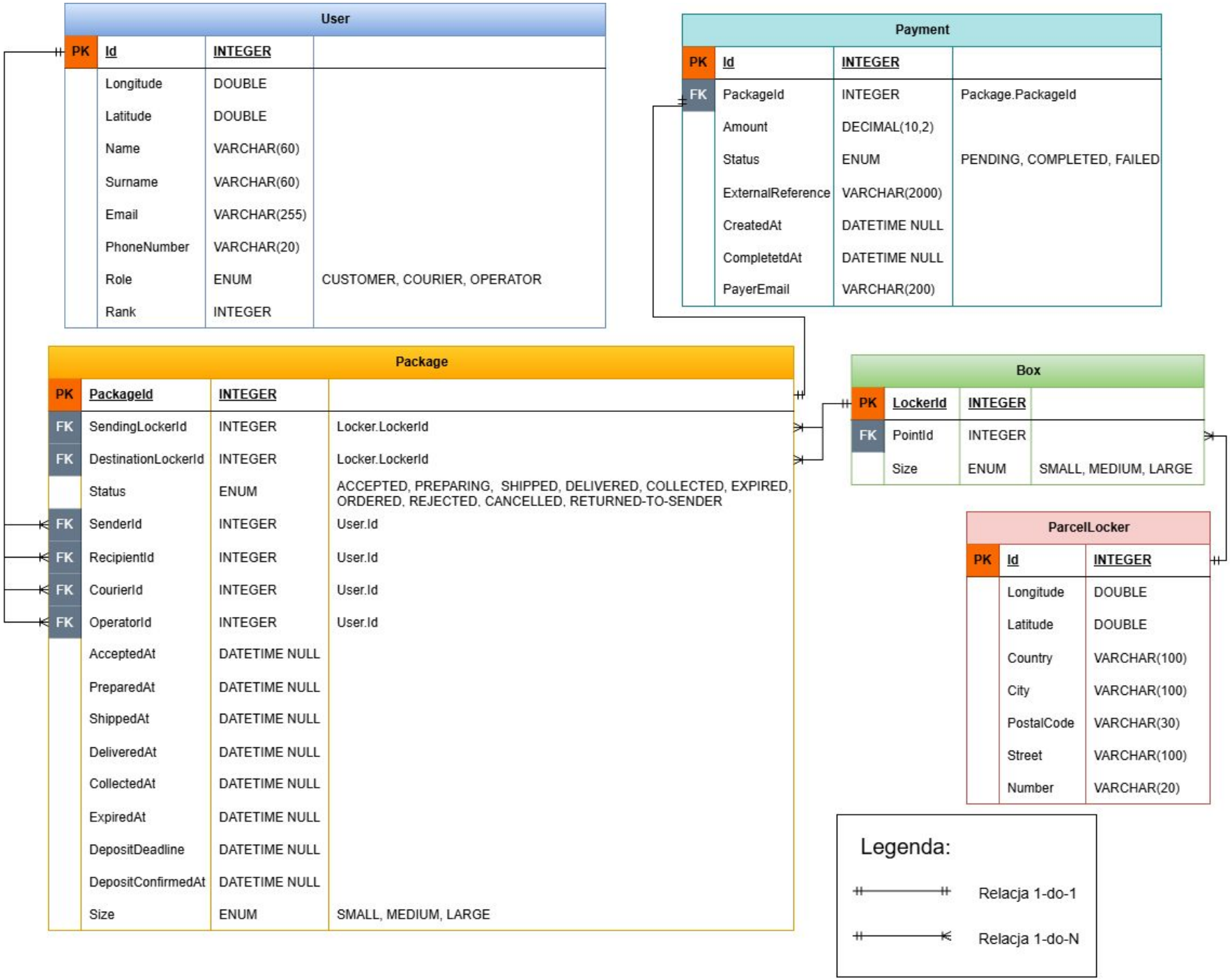
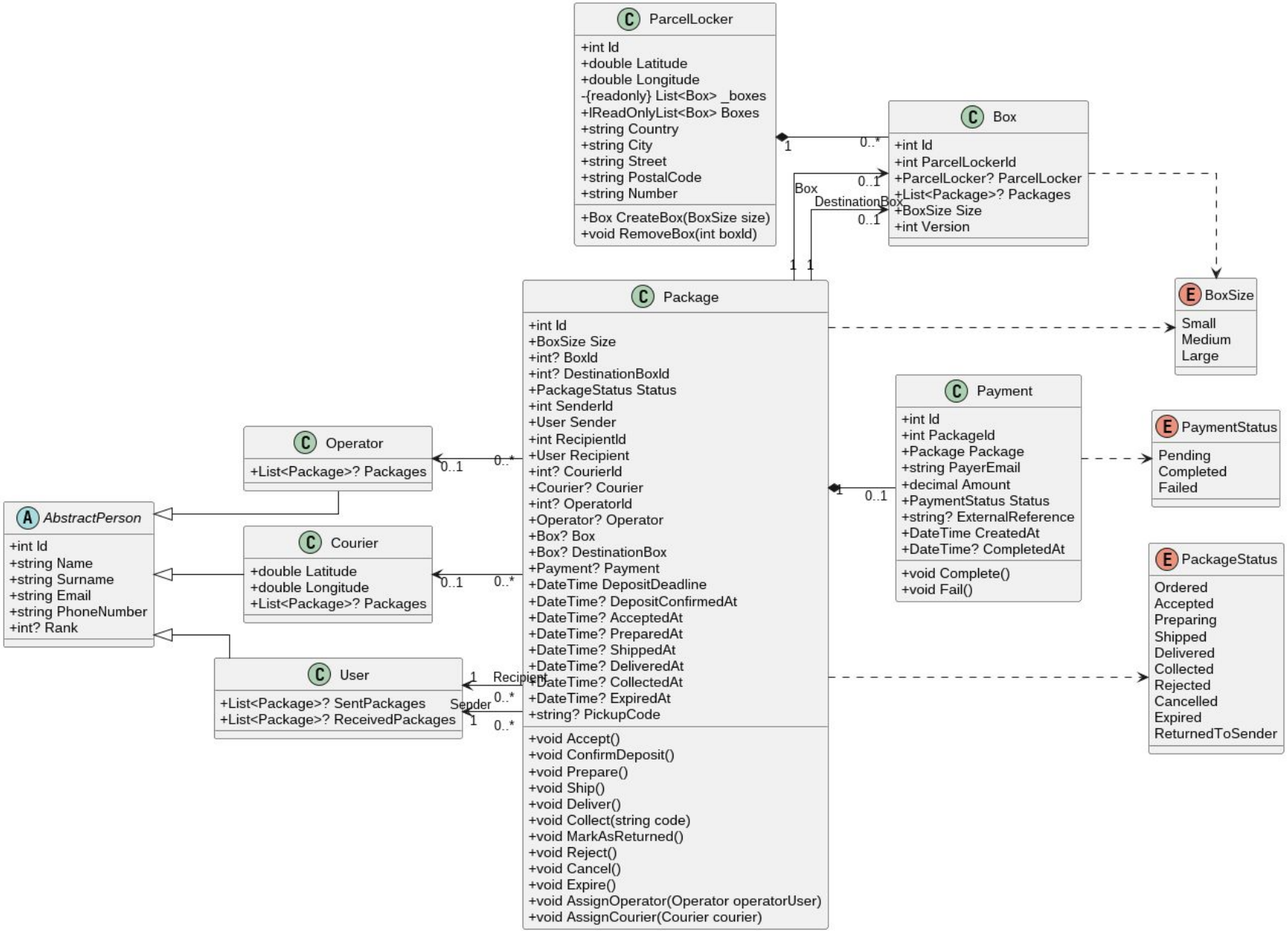

User
PK Id INTEGER
Longitude DOUBLE
Latitude DOUBLE
Name VARCHAR(60)
Surname VARCHAR(60)
Email VARCHAR(255)
PhoneNumber VARCHAR(20)
Role ENUM CUSTOMER, COURIER, OPERATOR
Rank INTEGER
Payment
PK Id INTEGER
FK PackageId INTEGER Package.PackageId
Amount DECIMAL(10,2)
Status ENUM PENDING, COMPLETED, FAILED
ExternalReference VARCHAR(2000)
CreatedAt DATETIME NULL
CompletetdAt DATETIME NULL
PayerEmail VARCHAR(200)
Package
PK PackageId INTEGER
FK SendingLockerId INTEGER Locker.LockerId
FK DestinationLockerId INTEGER Locker.LockerId
Status ENUM ACCEPTED, PREPARING, SHIPPED, DELIVERED, COLLECTED, EXPIRED, ORDERED, REJECTED, CANCELLED, RETURNED-TO-SENDER
FK SenderId INTEGER User.Id
FK RecipientId INTEGER User.Id
FK CourierId INTEGER User.Id
FK OperatorId INTEGER User.Id
AcceptedAt DATETIME NULL
PreparedAt DATETIME NULL
ShippedAt DATETIME NULL
DeliveredAt DATETIME NULL
CollectedAt DATETIME NULL
ExpiredAt DATETIME NULL
DepositDeadline DATETIME NULL
DepositConfirmedAt DATETIME NULL
Size ENUM SMALL, MEDIUM, LARGE
Box
PK LockerId INTEGER
FK PointId INTEGER
Size ENUM SMALL, MEDIUM, LARGE
ParcelLocker
PK Id INTEGER
Longitude DOUBLE
Latitude DOUBLE
Country VARCHAR(100)
City VARCHAR(100)
PostalCode VARCHAR(30)
Street VARCHAR(100)
Number VARCHAR(20)
Legenda:
Relacja 1-do-1
Relacja 1-do-N

Diagram klas



Opisy Metod (najważniejszych)

NOTE: Architektura jest w DDD. Część metod to metody domenowe ukryte podczas gdy realna logika i flow realizowana jest przez wyszczególnione serwisy.

`Package.Accept()` — operator przyjmuje paczkę. Przepuszcza tylko wtedy, gdy paczka jest zamówiona (`Ordered`), kurier (ten który akurat jest przy postlocker) potwierdził, że faktycznie leży w skrytce, i jest w całości opłacona. Każdy brakujący warunek rzuca wyjątkiem zamiast cichego niepowodzenia.

`Package.ConfirmDeposit()` — zapisuje, że paczka fizycznie trafiła do skrytki. Działa w 48h oknie od zamówienia; po tym czasie zamówienie uznaje się za nieaktualne.

`Package.Prepare() / Ship() / Deliver() / Collect()` — kolejne kroki podróży paczki (pakowanie → wysyłka → dostawa → odbiór). Każdy wymaga, żeby poprzedni krok się zakończył — nie da się „przeskoczyć" etapu.

`PackageAcceptanceService.AcceptPackageAsync` — znajduje paczkę i operatora, sprawdza że skrzynka pasuje rozmiarem, woła `Package.Accept()`, zapisuje do bazy i wysyła powiadomienie.

`PackageIntakeService.RequestPaymentAsync / ConfirmPaymentAsync` — uruchamia płatność u zewnętrznego dostawcy (podmienialnego, bo działa przez interfejs `IPaymentGateway`, obecnie zaiomplementowy Mock model w tym miejscu) i potem sprawdza jej wynik. Nie wymaga logowania — tylko e-mail płatnika.

`PackageIntakeService.ConfirmDepositAsync / ReportMissingDepositAsync` — kurier na miejscu potwierdza, że paczka jest w skrytce, albo zgłasza że jej nie ma (wtedy zamówienie od razu się anuluje (automatycznie bez udziału operatora), z winy nadawcy).

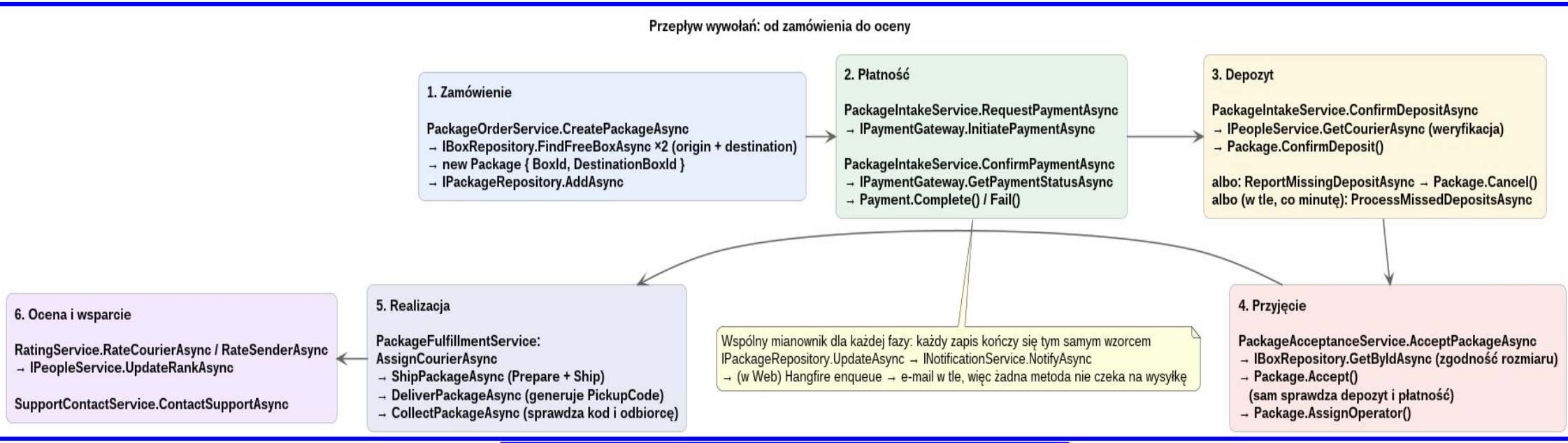
`PackageIntakeService.ProcessMissedDepositsAsync` — zadanie cykliczne (Hangfire), które co minutę sprawdza, czy ktoś nie zapomniał włożyć paczki w wyznaczonym czasie, i automatycznie anuluje takie zamówienia.

`PackageFulfillmentService` — wszystko po akceptacji: przydział kuriera, wysyłka, dostawa, odbiór. Każda metoda to: znajdź paczkę → zawołaj odpowiednią metodę domenową → zapisz → powiadom.

`RatingService.RateCourierAsync / RateSenderAsync` — odbiorca, dopiero po odebraniu paczki, ocenia kuriera i/lub nadawcę w skali 1–5. Ocena po prostu nadpisuje poprzednią wartość w profilu tej osoby.

`SupportContactService.ContactSupportAsync` — każdy user lub kurier może zgłosić problem; wiadomość leci tym samym kanałem powiadomień co reszta systemu (Hangfire → e-mail), bez osobnej infrastruktury.

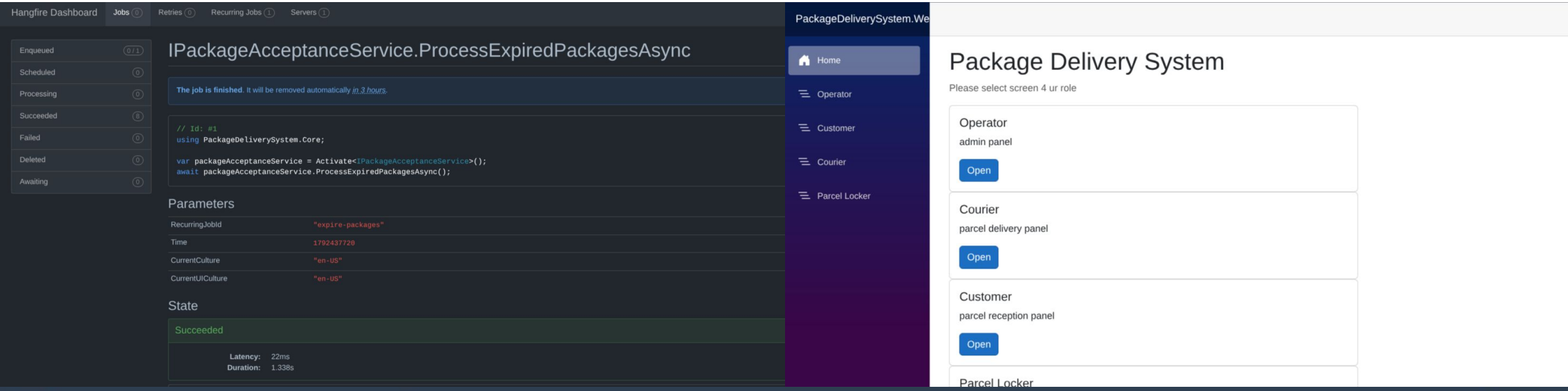
Code flow

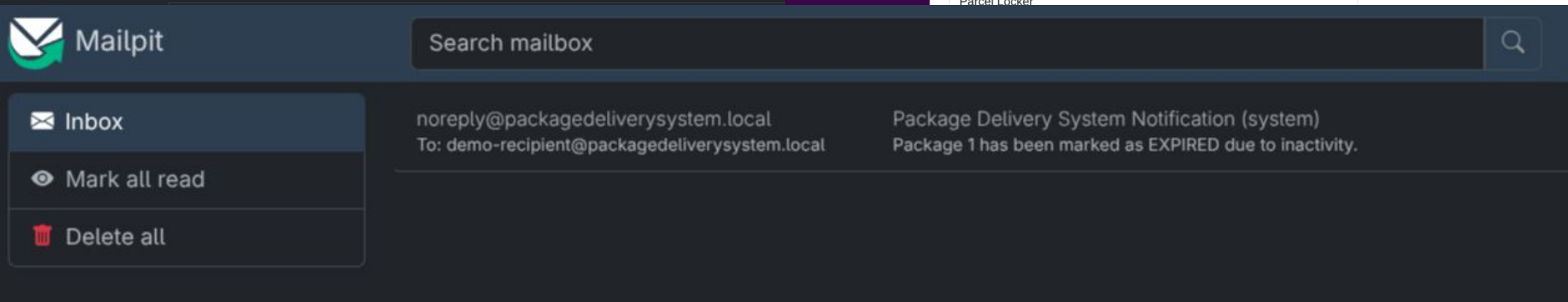

Przepływ wywołań: od zamówienia do oceny
1. Zamówienie
PackageOrderService.CreatePackageAsync
→ IBoxRepository.FindFreeBoxAsync ×2 (origin + destination)
→ new Package { BoxId, DestinationBoxId }
→ IPackageRepository.AddAsync
2. Płatność
PackageIntakeService.RequestPaymentAsync
→ IPaymentGateway.InitiatePaymentAsync
PackageIntakeService.ConfirmPaymentAsync
→ IPaymentGateway.GetPaymentStatusAsync
→ Payment.Complete() / Fail()
3. Depozyt
PackageIntakeService.ConfirmDepositAsync
→ IPeopleService.GetCourierAsync (weryfikacja)
→ Package.ConfirmDeposit()
albo: ReportMissingDepositAsync → Package.Cancel()
albo (w tle, co minutę): ProcessMissedDepositsAsync
4. Przyjęcie
PackageAcceptanceService.AcceptPackageAsync
→ IBoxRepository.GetByIdAsync (zgodność rozmiaru)
→ Package.Accept()
(sam sprawdza depozyt i płatność)
→ Package.AssignOperator()
5. Realizacja
PackageFulfillmentService:
AssignCourierAsync
→ ShipPackageAsync (Prepare + Ship)
→ DeliverPackageAsync (generuje PickupCode)
→ CollectPackageAsync (sprawdza kod i odbiorcę)
6. Ocena i wsparcie
RatingService.RateCourierAsync / RateSenderAsync
→ IPeopleService.UpdateRankAsync
SupportContactService.ContactSupportAsync
Wspólny mianownik dla każdej fazy: każdy zapis kończy się tym samym wzorcem
IPackageRepository.UpdateAsync → INotificationService.NotifyAsync
→ (w Web) Hangfire enqueue → e-mail w tle, więc żadna metoda nie czeka na wysyłkę



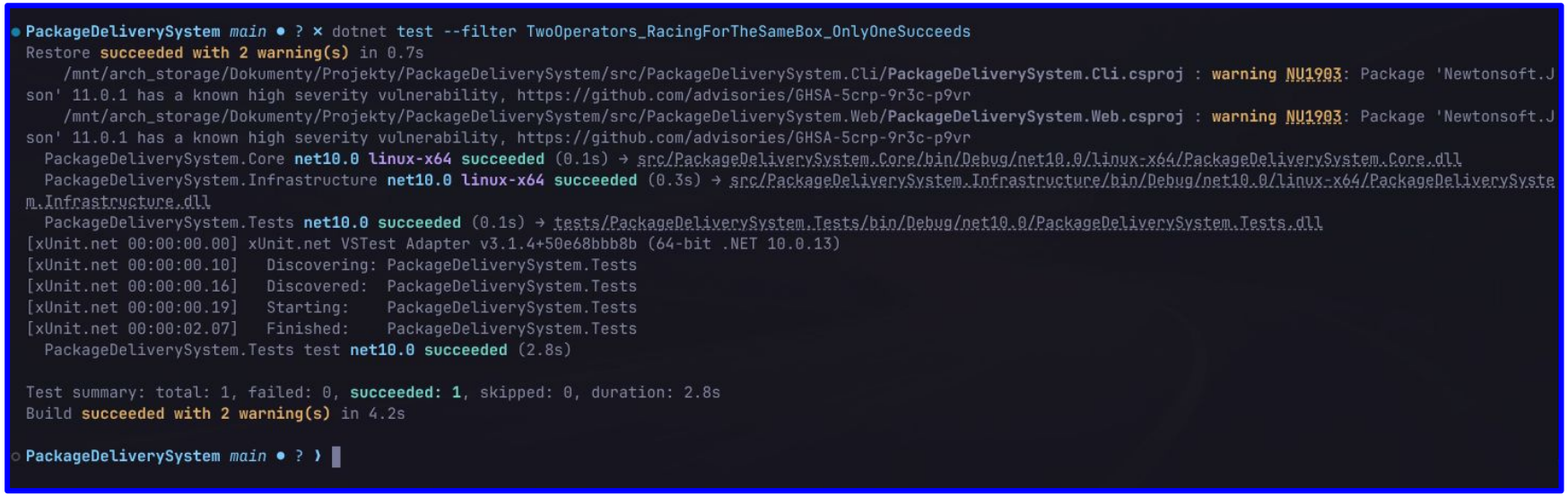
Mock-up’y

Screen paneli dla Operatora, Couriera, Customera (przykładowy) -> to framework więc dev w praktyce sam sobie zaimplementuje to w formie jaka będzie dla niego wygodna. Także panel MailPita po expire paczki oraz panel Hangfire z rutynowymi taskami :)





Testy (na szybkości)



Zrealizowano istotne z punktu widzenia code flow prototypy określające problemy mogące się pojawić podczas flow zamówienia. Race przy wkładaniu paczki, expire paczki i notyfikacje przy błędach przez MailPit. Zrealizowano też inne testy bardzo proste :)